ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



HỆ ĐIỀU HÀNH

BÁO CÁO ĐỒ ÁN

File & Process Management

Giảng viên hướng dẫn:	Phạm Tuấn Sơn	
	Lê Viết Long	
Sinh viên thực hiện:	Lê Hoàng Hiếu	24120048
	Đặng Hồng Minh	24120090
	Đinh Tiến Phát	24120405

Thành phố Hồ Chí Minh, 11/2026

Mục lục

1 Thông tin nhóm và Phân công công việc **1**

2 Mức độ hoàn thành **1**

3 Tìm hiểu mã nguồn xv6 **2**

3.1 Trường `ofile[]` của `struct proc` và `file[]` của `ftable` 2

3.2 Trace system call `open` 2

3.3 So sánh cách dùng file descriptor trong `open()`, `dup()` và `fork()` 3

3.4 Trace system call `fork` 4

3.5 Các trường `pid`, `state`, `context` của `struct proc` 4

3.6 Cơ chế lập lịch: `scheduler()`, `sched()`, `yield()` 5

4 Cài đặt system call `sysinfo` **6**

4.1 Cấu trúc `struct sysinfo` và luồng gọi 6

4.2 Thu thập danh sách open file 7

4.3 Thu thập danh sách process và bộ đếm `nsched` 8

4.4 Danh sách các file đã sửa đổi 8

5 Lựa chọn thiết kế và giả định **8**

6 Kết quả kiểm thử **9**

6.1 Cách kiểm thử 9

6.2 Tổng hợp các kịch bản 10

6.3 Một số kết quả tiêu biểu 11

7 Khó khăn và các cách đã thử **12**

8 Những khái niệm chưa hiểu rõ **13**

1 Thông tin nhóm và Phân công công việc

MSSV	Họ và tên	Nhiệm vụ	Mức độ đóng góp
24120048	Lê Hoàng Hiếu	– Tìm hiểu và báo cáo: so sánh cách dùng file descriptor trong `open()`, `dup()`, `fork()`; trace system call `fork`. – Viết chương trình kiểm thử `user/sysinfotest.c`. – Dựng khung báo cáo, viết mục thông tin nhóm, mức độ hoàn thành và kết quả kiểm thử.	100%
24120090	Đặng Hồng Minh	– Tìm hiểu và báo cáo: các trường `pid`, `state`, `context` của `struct proc`; cơ chế lập lịch trong `scheduler()`, `sched()`, `yield()`. – Đăng ký system call `sysinfo` (`syscall.h`, `syscall.c`, `usys.pl`, `user.h`, `Makefile`). – Thêm bộ đếm `nsched` vào `struct proc`, viết `proccollect()` thu thập danh sách process.	100%
24120405	Đinh Tiến Phát	– Tìm hiểu và báo cáo: trường `ofile[]` của `struct proc` và `file[]` của `ftable`; trace system call `open`. – Thiết kế `struct sysinfo` (`kernel/sysinfo.h`), viết `sys_sysinfo()`. – Viết `filecollect()`, `fileidx()`, `procs_using_file()` thu thập danh sách open file.	100%

Mỗi thành viên viết phần báo cáo tương ứng với phần mình tìm hiểu và cài đặt. Các mục 5, 7, 8 được ghép từ phần đóng góp của cả ba người.

2 Mức độ hoàn thành

Yêu cầu	Mục	Trạng thái
4.1. Tìm hiểu và báo cáo		
Trường `ofile[]` của `struct proc`, `file[]` của `struct ftable`	3.1	Hoàn thành
Trace system call `open` (luồng chính)	3.2	Hoàn thành
So sánh cách dùng file descriptor trong `open()`, `dup()` và `fork()`	3.3	Hoàn thành
Trace system call `fork` (luồng chính)	3.4	Hoàn thành
Các trường `pid`, `state`, `context` của `struct proc`	3.5	Hoàn thành
Cơ chế lập lịch trong `scheduler()`, `sched()`, `yield()`	3.6	Hoàn thành
4.2. Cài đặt và báo cáo		
System call `sysinfo` nhận con trỏ tới `struct sysinfo` (định nghĩa trong `kernel/sysinfo.h`), thông tin được điền trong kernel rồi chép về user space	4.1	Hoàn thành
Số open file của toàn hệ thống và danh sách kèm loại, chế độ mở, reference count, danh sách process tham chiếu tới file	4.2	Hoàn thành
Số process có trạng thái khác `UNUSED` và danh sách kèm pid, trạng thái, pid của cha, số lần được lập lịch, danh sách open file	4.3	Hoàn thành
Chương trình `user/sysinfotest.c` gọi `sysinfo` nhiều lần để kiểm thử nhiều kịch bản (17 kịch bản, 118 kiểm tra, tất cả đều đạt)	6	Hoàn thành

3 Tìm hiểu mã nguồn xv6

3.1 Trường `ofile[]` của `struct proc` và `file[]` của `ftable`

xv6 quản lý file đang mở theo ba tầng: bảng fd riêng của mỗi process, bảng `struct file` chung của cả hệ thống, và tài nguyên bên dưới (inode, pipe hoặc thiết bị).

`ofile[]` của `struct proc` (`kernel/proc.h`) là mảng `NOFILE` = 16 con trỏ `struct file *`, riêng cho từng process. Chỉ số của mảng chính là số fd mà chương trình user nhận được: `ofile[fd]` trỏ tới `struct file` của fd đó, hoặc bằng 0 nếu fd chưa dùng. Theo quy ước, fd 0, 1, 2 là input, output và error chuẩn. Vì vậy mỗi process mở được tối đa 16 file cùng lúc.

`file[]` của `ftable` (`kernel/file.c`) là mảng `NFILE` = 100 `struct file` dùng chung cho cả hệ thống, được bảo vệ bởi spinlock `ftable.lock`. `filealloc()` tìm ô có `ref == 0` và đặt `ref = 1`; `filedup()` tăng `ref`; `fileclose()` giảm `ref` và chỉ thật sự đóng file khi `ref` về 0. Mỗi `struct file` (`kernel/file.h`) có các trường:

- `type`: `FD_NONE` (ô trống), `FD_PIPE`, `FD_INODE` (file hoặc thư mục trên đĩa) hoặc `FD_DEVICE` (thiết bị, ví dụ console);
- `ref`: số con trỏ trong các `ofile[]` đang trỏ tới nó;
- `readable`, `writable`: quyền đọc, ghi, lấy từ chế độ mở;
- `pipe`, `ip`: con trỏ tới `struct pipe` hoặc `struct inode` bên dưới;
- `off`: vị trí đọc/ghi hiện tại, dùng cho `FD_INODE`;
- `major`: số hiệu thiết bị, dùng để tra bảng `devsw[]` khi là `FD_DEVICE`.

Tầng giữa quyết định cái gì được dùng chung. Hai fd trỏ tới cùng một `struct file` (sau `dup` hoặc `fork`) thì dùng chung `off`. Hai lần `open` cùng một đường dẫn cho hai `struct file` khác nhau, mỗi cái có `off` riêng, nhưng cùng trỏ tới một inode.

3.2 Trace system call `open`

Vào kernel. Hàm `open(path, omode)` ở user là một stub trong `user/usys.S` (sinh từ `usys.pl`): nó nạp `SYS_open` (bằng 15) vào thanh ghi `a7` rồi thực hiện `ecall`. CPU chuyển sang S-mode và nhảy tới `uservec` trong trang trampoline; `uservec` lưu các thanh ghi user vào `p->trapframe`, chuyển sang kernel stack và gọi `usertrap()`. `usertrap()` thấy `scause = 8` (syscall từ U-mode), cộng 4 vào `epc` để khi quay về sẽ chạy lệnh sau `ecall`, rồi gọi `syscall()`. `syscall()` đọc số syscall trong `a7` và gọi `sys_open()` qua bảng `syscalls[]`.

Trong `sys_open()` (`kernel/sysfile.c`):

1. Lấy `omode` bằng `argint(1, ...)` và chép đường dẫn vào kernel bằng `argstr(0, path, MAXPATH)`.
2. `begin_op()` mở một giao dịch log của hệ thống file, vì `open` có thể tạo mới hoặc cắt ngắn file.
3. Tìm inode. Có `O_CREATE` thì `create()` tìm hoặc tạo file và trả về inode đã khóa. Không có thì `namei()` phân giải đường dẫn rồi `ilock()`; thư mục chỉ được mở với `O_RDONLY`.

4. Với file thiết bị, kiểm tra `major` nằm trong $[0, \texttt{NDEV})$.

5. `filealloc()` lấy một `struct file` trống trong `ftable` (`ref = 1`), rồi `fdalloc()` gắn nó vào ô trống có chỉ số nhỏ nhất của `p->ofile[]`.

6. Điền `struct file`: `type` là `FD_DEVICE` (kèm `major`) hoặc `FD_INODE` (kèm `off = 0`); `ip`; `readable` khi không có `O_WRONLY`; `writable` khi có `O_WRONLY` hoặc `O_RDWR`. Có `O_TRUNC` với file thường thì `itrunc()` xóa nội dung.

7. `iunlock()` (inode vẫn được giữ qua `f->ip`), `end_op()`, trả về `fd`. `syscall()` ghi giá trị này vào `trapframe->a0`, là giá trị trả về mà chương trình user nhận được.

Đường lỗi. Bước nào lỗi cũng trả về -1 sau khi hoàn tác những gì đã làm. Chưa có inode thì chỉ cần `end_op()`; đã khóa inode (thư mục mở để ghi, thiết bị sai) thì `iunlockput()` rồi `end_op()`. Đáng chú ý nhất là khi `filealloc()` thành công nhưng `fdalloc()` thất bại vì process đã dùng hết 16 fd: kernel phải `fileclose(f)` để trả lại ô trong `ftable`, nếu không ô đó bị mất vĩnh viễn.

3.3 So sánh cách dùng file descriptor trong `open()`, `dup()` và `fork()`

fd chỉ là chỉ số trong `ofile[]`, trỏ tới một `struct file` trong `ftable` (mục 3.1). Ba lời gọi khác nhau ở chỗ có tạo `struct file` mới hay chỉ thêm con trỏ tới một `struct file` đã có:

- `open()` gọi `filealloc()` để tạo `struct file` mới (`ref = 1`, `off = 0`) rồi `fdalloc()` để cấp fd. Mở cùng một file hai lần cho hai fd có offset độc lập, chỉ chung inode.
- `dup(fd)` cấp fd mới bằng `fdalloc()`, trỏ tới *chính* `struct file` của fd cũ, rồi `filedup()` tăng `ref`. Hai fd dùng chung offset và quyền đọc/ghi.
- `fork()` gán `np->ofile[i] = filedup(p->ofile[i])` cho mọi fd của cha. Con có đúng các số fd của cha, trỏ tới cùng các `struct file`, nên cha và con dùng chung offset; mỗi fd được chép làm `ref` tăng 1.

Chiều ngược lại, `close()` gọi `fileclose()` để giảm `ref`. Chỉ khi `ref` về 0 thì `struct file` mới được giải phóng và tài nguyên bên dưới được trả lại (`iput()` hoặc `pipeclose()`). `exit()` cũng đóng mọi fd theo cách này.

	`open()`	`dup()`	`fork()`
Tạo `struct file` mới	Có (`filealloc`)	Không	Không
fd được tạo	1 fd, ô trống nhỏ nhất	1 fd, ô trống nhỏ nhất	Chép mọi fd của cha sang con, giữ nguyên số fd
`ref`	`struct file` mới có `ref = 1`	Tăng 1	Tăng 1 cho mỗi fd được chép
Offset	Riêng, bắt đầu từ 0	Chung với fd gốc	Chung giữa cha và con
Quyền đọc/ghi	Theo `omode`	Giống fd gốc	Giống của cha
Process giữ fd	Process gọi `open`	Process gọi `dup`	Cả cha và con

Bảng 1: So sánh cách dùng file descriptor trong `open()`, `dup()` và `fork()`

Việc `fork()` dùng chung offset là có chủ ý: shell dựa vào nó để các lệnh con ghi nối tiếp vào cùng một file khi chuyển hướng output [1]. Test 3, 4, 5 ở mục 6 kiểm chứng các tính chất trên. Ngay cả `ref = 9` của console lúc hệ thống mới chạy cũng giải thích được: `init` mở console rồi `dup` hai lần (`ref = 3`), sau đó `init` fork ra `sh` và `sh` fork ra `sysinfotest`, mỗi lần fork cộng thêm 3.

3.4 Trace system call `fork`

1. **Vào kernel** giống `open` (mục 3.2): stub `fork` trong `usys.S`, `ecall`, `usertrap()`, `syscall()`, rồi `sys_fork()` gọi `fork()` trong `kernel/proc.c`.
2. `allocproc()` tìm một ô `UNUSED` trong `proc[]`, cấp pid mới bằng `allocpid()`, đặt trạng thái `USED` (và `nsched = 0`, phần nhóm thêm), cấp trang trapframe và page table, rồi đặt `context.ra = forkret` và `context.sp` ở đỉnh kernel stack. Hàm trả về khi vẫn giữ `np->lock`. Hết ô hoặc hết bộ nhớ thì `fork()` trả về -1.
3. `uvmcopy()` chép toàn bộ bộ nhớ user của cha sang các trang mới của con (xv6 không dùng copy-on-write). Lỗi thì `freeproc(np)` và trả về -1.
4. **Chép trapframe** của cha sang con, rồi gán `np->trapframe->a0 = 0`.
5. **Chia sẻ tài nguyên**: `filedup()` mọi fd đang mở (mục 3.3), `idup()` thư mục hiện hành, chép tên process.
6. **Gắn cha và cho chạy**: nhả `np->lock`, lấy `wait_lock` để gán `np->parent = p`, rồi lấy lại `np->lock` để đặt `RUNNABLE`. Phải nhả `np->lock` trước vì xv6 luôn lấy `wait_lock` trước `p->lock`.
7. **Trả về pid của con**; `syscall()` ghi giá trị này vào `a0` của cha.

Vì sao cha nhận pid của con, còn con nhận 0. Con quay về user với bản sao trapframe của cha, nên chạy tiếp từ cùng lệnh sau `ecall`; chỉ khác thanh ghi `a0` chứa giá trị trả về. `a0` của cha được `syscall()` ghi bằng pid của con, còn `a0` của con đã bị `fork()` gán 0 và con không đi qua `syscall()`.

Lần chạy đầu tiên của con. Khi `scheduler()` chọn con, `swtch()` nạp `context` của con và lệnh `ret` nhảy tới `forkret()` (vì `ra = forkret`). `forkret()` nhả `p->lock` mà scheduler đang giữ, rồi `usertrapret()` đưa con về user mode.

Test 5 (mục 6) cho thấy ngay sau `fork`, con có `ppid` bằng pid của cha, có cùng các fd, và ở trạng thái `RUNNABLE` với `nsched = 0` vì chưa được chạy lần nào.

3.5 Các trường `pid`, `state`, `context` của `struct proc`

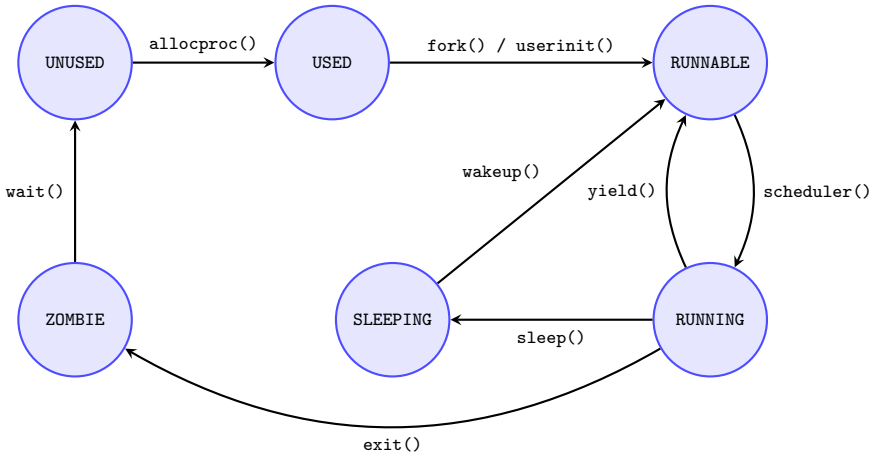
Mỗi process là một `struct proc` (`kernel/proc.h`) trong bảng `proc[NPROC]`, với `NPROC` $= 64$.

`pid` được gán trong `allocproc()` bằng `allocpid()`. Hàm này tăng biến toàn cục `nextpid` dưới khóa `pid_lock`, nên mỗi process nhận một pid riêng kể cả khi nhiều CPU cùng fork. `nextpid` chỉ tăng, pid của process đã kết thúc không được dùng lại.

`state` nhận một trong 6 giá trị của `enum procstate`, chuyển đổi như Hình 1:

- `UNUSED`: ô trống; `USED`: vừa được `allocproc()` lấy, đang khởi tạo;
- `RUNNABLE`: sẵn sàng chạy, chờ CPU; `RUNNING`: đang chạy trên một CPU;
- `SLEEPING`: đang chờ một sự kiện trong `sleep()`; `ZOMBIE`: đã `exit()` nhưng cha chưa `wait()`.

Đọc hay ghi `state` đều phải giữ `p->lock`. Nếu không, hai CPU có thể cùng thấy một process `RUNNABLE` và cùng chạy nó.



Hình 1: Các trạng thái của process và hàm gây ra mỗi lần chuyển

`context` chứa các thanh ghi `ra`, `sp` và `s0`–`s11` của kernel thread lúc nó rời CPU. `swtch()` (`kernel/swtch.S`) lưu các thanh ghi này vào context cũ và nạp từ context mới, nên lệnh `ret` cuối `swtch()` nhảy tới `ra` của thread mới. Chỉ cần lưu thanh ghi callee-saved vì `swtch()` được gọi như một hàm C, các thanh ghi còn lại đã được trình biên dịch lưu trên stack. `context` khác với `trapframe`:

	`context`	`trapframe`
Lưu gì	`ra`, `sp`, `s0`–`s11` của kernel thread	Toàn bộ 32 thanh ghi user và `epc`
Khi nào	Chuyển giữa process và scheduler (`swtch()`)	Mỗi lần trap từ user vào kernel (`uservec`, `userret`)
Nằm ở đâu	Trong `struct proc` (của scheduler thì trong `struct cpu`)	Một trang riêng, map ngay dưới trang trampoline

Bảng 2: So sánh `context` và `trapframe`

3.6 Cơ chế lập lịch: `scheduler()`, `sched()`, `yield()`

Mỗi CPU có một kernel thread riêng chạy `scheduler()` (`kernel/proc.c`) trong một vòng lặp vô hạn:

1. Bật ngắt (`intr_on()`), để tránh deadlock khi mọi process đều đang chờ ngắt thiết bị.
2. Duyệt `proc[]` theo vòng tròn (round-robin). Với mỗi `p`, lấy `p->lock`; nếu `p` đang `RUNNABLE` thì đặt `RUNNING`, ghi `c->proc = p`, tăng `p->nsched` (phần nhóm thêm), rồi `swtch(&c->context, &p->context)` để chạy `p`.

3. Khi `p` nhường CPU, `swtch` quay về đây: đặt `c->proc = 0`, nhả `p->lock`, xét process tiếp theo.

4. Duyệt hết bảng mà không có process nào để chạy thì thực hiện lệnh `wfi` để chờ ngắt.

`sched()` là đường duy nhất để một process trả CPU về scheduler: nó gọi `swtch(&p->context, &mycpu()->context)`, có lưu và khôi phục `intena` của CPU. Trước đó hàm kiểm tra bốn điều kiện, sai điều nào thì `panic`:

- Phải đang giữ `p->lock`, để CPU khác không chọn chạy `p` khi `p` chưa lưu xong context.
- Chỉ giữ đúng một khóa (`noff == 1`). Nhường CPU khi còn giữ khóa khác sẽ làm các process cần khóa đó phải quay chờ mãi.
- `state` đã khác `RUNNING`: người gọi phải đặt trạng thái mới trước (`RUNNABLE`, `SLEEPING` hoặc `ZOMBIE`).
- Ngắt đang tắt, để không có ngắt nào chen vào giữa lúc chuyển context.

`yield()` lấy `p->lock`, đặt `RUNNABLE` rồi gọi `sched()`; khi được chạy lại thì nhả khóa. `usertrap()` và `kerneltrap()` gọi `yield()` mỗi khi gặp ngắt timer (`which_dev == 2`), nên một process chạy vòng lặp vô hạn vẫn bị lấy lại CPU sau mỗi tick.

`p->lock` được giữ qua `swtch`. Khóa này được lấy ở một thread và nhả ở thread khác. Scheduler lấy `p->lock` rồi chuyển sang `p`, và chính `p` nhả khóa (trong `yield()`, `sleep()`, hoặc `forkret()` nếu là lần chạy đầu). Ngược lại, `p` lấy khóa trước khi gọi `sched()`, và scheduler nhả nó sau khi `swtch` quay về. Nhờ vậy trạng thái của `p` được bảo vệ suốt quá trình chuyển, không CPU nào chen vào giữa được.

4 Cài đặt system call `sysinfo`

4.1 Cấu trúc `struct sysinfo` và luồng gọi

`struct sysinfo` và hai struct con được định nghĩa trong `kernel/sysinfo.h` (Bảng 3). Process và file được nối với nhau qua chỉ số trong `ftable`: fd thứ k của một process trỏ tới file có `idx` bằng `fidx[`k`]`.

Kích thước và cách chép ra user. Một `sysinfo_file` chiếm 280 byte, một `sysinfo_proc` chiếm 164 byte, nên cả `struct sysinfo` là $4 + 100 \times 280 + 4 + 64 \times 164 = 38\,504$ byte, khoảng 38.5 KB. Kernel stack của xv6 chỉ có một trang 4 KB, nên kernel không dựng cả struct trên stack. Mỗi lần kernel chỉ dựng một phần tử (vài trăm byte) trên stack rồi `copyout()` thẳng vào đúng chỗ trong buffer của user: phần tử thứ k của `files` nằm ở `addr` cộng offset của trường `files` (tính bằng `__builtin_offsetof`) cộng $k \times 280$; mảng `procs` tính tương tự. User stack cũng chỉ có một trang (`USERSTACK = 1`), nên chương trình user phải khai báo `struct sysinfo` toàn cục hoặc cấp bằng `malloc`.

Luồng gọi. `sysinfo(&info)` ở user đi qua `ecall`, `usertrap()`, `syscall()` tới `sys_sysinfo()` (`kernel/sysproc.c`). Hàm này lấy địa chỉ buffer bằng `argaddr(0, &addr)`, gọi `filecollect(addr)` (mục 4.2) rồi `proccollect(addr)` (mục 4.3), mỗi hàm tự ghi mảng của mình và trả về số phần tử, cuối cùng `copyout` hai số `nfile` và `nproc`. Nếu có `copyout` nào lỗi (địa chỉ user không hợp lệ) thì syscall trả về -1, ngược lại trả về 0.

Trường	Ý nghĩa
`struct sysinfo`	
`nfile`, `files[NFILE]`	Số file đang mở trong toàn hệ thống và danh sách của chúng
`nproc`, `procs[NPROC]`	Số process có trạng thái khác `UNUSED` và danh sách của chúng
`struct sysinfo_file`: một file đang mở	
`idx`	Vị trí trong `ftable.file[]`, dùng để nối với process
`type`	Loại file: `FD_PIPE`, `FD_INODE` hoặc `FD_DEVICE`
`readable`, `writable`	Chế độ mở
`ref`	Reference count
`npids`, `pids[NPROC]`	Các process có ít nhất một fd trỏ tới file này
`struct sysinfo_proc`: một process	
`pid`, `state`	pid và trạng thái (giá trị của `enum procstate`)
`ppid`	pid của cha, bằng 0 nếu không có cha (`init`)
`nsched`	Số lần được scheduler chọn chạy
`name[16]`	Tên process
`nfd`, `fds[NOFILE]`, `fidx[NOFILE]`	Số fd đang mở, các số fd, và `idx` của file mà mỗi fd trỏ tới

Bảng 3: Các trường của `struct sysinfo`

4.2 Thu thập danh sách open file

`fileidx(f)` (`kernel/file.c`) trả về vị trí của `f` trong `ftable.file[]`, tức `f - ftable.file`, hoặc -1 nếu `f` không thuộc bảng. `ftable` được khai báo bằng struct ẩn danh nên chỉ dùng được trong `file.c`; `proccollect()` ở `proc.c` cần hàm này để đổi con trỏ trong `ofile[]` thành `idx`.

`procs_using_file(f, pids)` (`kernel/proc.c`) duyệt `proc[]`. Với mỗi process khác `UNUSED`, hàm giữ `p->lock` trong lúc quét `ofile[]` tìm ô trỏ tới `f`, và dừng ngay ở ô đầu tiên tìm thấy. Nhờ vậy một process có nhiều fd tới cùng một file (sau `dup`) chỉ được ghi pid một lần, và số pid không vượt quá `NPROC`, vừa với mảng `pids`.

`filecollect(addr)` (`kernel/file.c`) duyệt từng ô i của `ftable.file[]`:

1. Lấy `ftable.lock`. Ô có `ref == 0` thì nhả khóa và bỏ qua.
2. Xóa biến cục bộ `sf` bằng `memset` (để các phần tử thừa của `pids` không mang dữ liệu cũ trên kernel stack ra user), chép `idx = i`, `type`, `readable`, `writable`, `ref`, rồi nhả `ftable.lock`.
3. Gọi `procs_using_file()` để điền `npids` và `pids`.
4. `copyout` `sf` vào phần tử thứ `count` của `files` rồi tăng `count`; `copyout` lỗi thì trả về -1.

Cuối cùng hàm trả về `count`, là số file đang mở.

Khóa. `ftable.lock` được nhả trước bước 3 và 4 vì hai lý do. Thứ nhất, xv6 không quy định thứ tự giữa `ftable.lock` và `p->lock`, nên nhóm không bao giờ giữ cả hai cùng lúc để khỏi lo deadlock. Thứ hai, `copyout()` phải đi qua page table của user và có thể chậm, không nên làm trong lúc giữ spinlock (ngắt bị tắt, các CPU khác cần khóa phải quay chờ). Cái giá phải trả là kết quả không nhất quán tuyệt đối, xem mục 5.

4.3 Thu thập danh sách process và bộ đếm `nsched`

Bộ đếm `nsched`. Thêm trường `int nsched` vào `struct proc`, trong nhóm các trường được bảo vệ bởi `p->lock`. `allocproc()` đặt nó về 0 khi cấp ô mới, và `scheduler()` tăng nó ngay trước `swtch()` khi chọn process để chạy (mục 3.6). Cả hai chỗ đều đang giữ `p->lock`, nên không có race giữa các CPU. Vì sao đếm ở chỗ này được giải thích ở mục 5.

`proccollect(addr)` (`kernel/proc.c`) duyệt từng ô `p` của `proc[]`:

1. Lấy `wait_lock` rồi `p->lock`. Ô `UNUSED` thì nhả hai khóa và bỏ qua.
2. Xóa biến cục bộ `sproc`, chép `pid`, `state`, `nsched`, `name`; `ppid` lấy từ `p->parent->pid`, hoặc bằng 0 nếu không có cha (chỉ có `init`).
3. Duyệt `ofile[]`: với mỗi fd đang mở, ghi số fd vào `fds` và `fileidx(ofile[fd])` vào `fidx`.
4. Nhả cả hai khóa, rồi `copyout` `sproc` vào phần tử thứ `count` của `procs`; lỗi thì trả về -1.

Cuối cùng hàm trả về `count`, là số process khác `UNUSED`.

Thứ tự khóa. `p->parent` được bảo vệ bởi `wait_lock`. Trong `exit()` và `wait()`, xv6 luôn lấy `wait_lock` trước `p->lock`; nếu `proccollect()` lấy theo thứ tự ngược lại thì có thể deadlock với một process đang `exit()`. Giống `filecollect()`, `copyout()` chỉ được gọi sau khi đã nhả hết khóa.

4.4 Danh sách các file đã sửa đổi

File	Thay đổi	Người
`kernel/sysinfo.h`	Viết lại: `struct sysinfo_file`, `sysinfo_proc`, `sysinfo`	A
`kernel/sysproc.c`	`sys_sysinfo()`	A
`kernel/file.c`	`filecollect()`, `fileidx()`	A
`kernel/proc.h`	Thêm trường `nsched` vào `struct proc`	C
`kernel/proc.c`	Khởi tạo và tăng `nsched`, `proccollect()` (C); `procs_using_file()` (A)	A, C
`kernel/defs.h`	Khai báo các hàm mới ở trên	A, C
`kernel/syscall.h`, `syscall.c`	`SYS_sysinfo` = 22, thêm `sys_sysinfo` vào `syscalls[]`	C
`user/usys.pl`, `user/user.h`	Stub `sysinfo`; khai báo `struct sysinfo;` và `int sysinfo(struct sysinfo *);`	C
`Makefile`	Thêm `$U/_sysinfotest` vào `UPROGS`	C
`user/sysinfotest.c`	Viết lại toàn bộ: 17 kịch bản kiểm thử (mục 6)	B

Bảng 4: Các file đã sửa đổi

5 Lựa chọn thiết kế và giả định

- **Kết quả không phải là một bản chụp tức thời của cả hệ thống.** `filecollect()` chạy trước `proccollect()`, và mỗi ô chỉ bị khóa trong lúc chép. Nếu có process khác mở, đóng file hay thoát cùng

lúc, hai danh sách có thể không khớp nhau, ví dụ một file vẫn liệt kê pid của process vừa đóng nó (mục 7 có một trường hợp thật). Muốn chụp nhất quán thì phải giữ cùng lúc `ftable.lock` và khóa của mọi process, rất dễ deadlock. Nhóm chấp nhận điều này vì `sysinfo` chỉ dùng để quan sát.

- **Đọc `ofile[]` của process khác.** `ofile[]` chỉ do chính process đó sửa và không có khóa riêng; giữ `p->lock` chỉ bảo đảm process không bị giải phóng trong lúc đọc. Con trỏ đọc được có thể vừa bị `close`, nhưng kernel chỉ so sánh con trỏ (`procs_using_file()`) hoặc tính chỉ số từ nó (`fileidx()`), không truy cập vào `struct file`, nên không có nguy cơ đọc vùng nhớ đã giải phóng.

- **Test so sánh trước và sau thao tác.** Các giá trị như `ref` của console, pid hay `nsched` phụ thuộc vào `init`, `sh` và thời điểm chạy, nên mỗi test chụp `sysinfo` trước và sau thao tác rồi kiểm tra độ chênh lệch (ví dụ `nfile` tăng đúng 5 sau 5 lần `open`). Chỉ các hằng số (`NOFILE`, `NFILE`, `NPROC`, `init` có pid 1) mới được so sánh trực tiếp. Cuối mỗi test luôn kiểm tra `nfile`, `nproc` đã trở về như lúc đầu.

- **Chỉ đối chiếu hai danh sách khi hệ thống đứng yên.** Do kết quả không phải bản chụp tức thời (ý đầu tiên của mục này), các kiểm tra đối chiếu danh sách file với danh sách process chỉ được làm khi mọi process khác đang bị chặn. Khi chờ một process đổi trạng thái, test chụp thêm một lần sau khi thấy trạng thái đó rồi mới đối chiếu.

- **Giữ process con đứng yên bằng pipe.** Con bị chặn ở `read()` trên một pipe rỗng; cha ghi 1 byte khi muốn con chạy tiếp và thoát. Cách này chắc chắn hơn chờ bằng `sleep`. Khi cần chờ một trạng thái (`ZOMBIE`, `SLEEPING`), test gọi `sysinfo` liên tục tới khi thấy, tối đa 100 tick.

- **Không phụ thuộc số CPU.** Test 10 chỉ yêu cầu các con chạy vòng lặp vô hạn luôn ở `RUNNING` hoặc `RUNNABLE` và có lúc thấy `RUNNABLE`; với 1 CPU thì CPU đó đang chạy chính `sysinfotest` nên không con nào `RUNNING`. Nhóm đã chạy với `CPUS=3` và `CPUS=1`, đều đạt.

- **Định nghĩa "scheduling event".** Đề không định nghĩa rõ. Nhóm quy ước mỗi lần process được `scheduler()` chọn và `swtch()` vào chạy là một sự kiện, nên `nsched` chỉ được tăng ở đúng một chỗ, ngay trước `swtch()`. Mỗi lần như vậy process thật sự nhận CPU. Đếm theo lần nhường CPU thì phải sửa nhiều chỗ (`yield`, `sleep`, `exit`) mà không thêm ý nghĩa gì.

6 Kết quả kiểm thử

6.1 Cách kiểm thử

Chương trình `user/sysinfotest.c` gọi `sysinfo` trước và sau mỗi thao tác (`open`, `dup`, `fork`, `exit`, ...) rồi so sánh hai lần chụp; mỗi kiểm tra in `[PASS]` hoặc `[FAIL]` kèm giá trị đo được. Chạy `sysinfotest` để chạy cả 17 test, `sysinfotest 5 8` để chỉ chạy một số test, hoặc `sysinfotest show` để chỉ in hai danh sách hiện tại.

Ngoài kiểm tra riêng của từng kịch bản, nhiều test còn kiểm tra hai tính chất chung: danh sách file và danh sách process phải khớp nhau (pid trong danh sách của một file phải có fd trỏ tới file đó, và ngược lại), và `ref` của mỗi file phải bằng tổng số fd trỏ tới nó.

Nhóm chạy trên QEMU ba lần với 3 CPU (mặc định của `make qemu`) và một lần với `CPUS=1`. Cả bốn lần đều đạt 118/118 kiểm tra và kernel không panic.

6.2 Tổng hợp các kịch bản

#	Kịch bản	Nội dung kiểm tra chính	Số KT
1	Trạng thái ban đầu	Có `init` (pid 1, ppid 0), `sh` đang `SLEEPING` trong `wait`, `sysinfotest` đang `RUNNING`; fd 0, 1, 2 cùng trỏ console (`DEVICE`, `rw`); hai danh sách khớp nhau	13
2	`open` với từng chế độ	`O_RDONLY`, `O_WRONLY`, `O_RDWR` cho `INODE` với quyền `r-`, `-w`, `rw`; mở thư mục cho `INODE r-`; mở lại console cho một entry `DEVICE` mới	10
3	`open` một file hai lần	Hai entry khác nhau, mỗi entry `ref = 1`, offset độc lập	6
4	`dup`	Không thêm entry, `ref` tăng lên 2, pid chỉ có một lần, offset dùng chung	6
5	`fork`	`nproc` tăng 1, con có `ppid` đúng và cùng fd với cha, `ref` tăng theo số fd được chép, offset dùng chung, sau `wait` mọi thứ trở lại như cũ	14
6	`pipe`	Hai entry `PIPE`: đầu đọc `r-`, đầu ghi `-w`	4
7	`close`	Đóng một trong hai fd: `ref` giảm còn 1; đóng fd cuối: entry biến mất	7
8	Process `ZOMBIE`	Con đã `exit` nhưng cha chưa `wait`: trạng thái `ZOMBIE`, không còn fd; sau `wait` con biến mất, mã thoát đúng	9
9	Process `SLEEPING`, `kill`	Con chờ `read` trên pipe rỗng và con trong `sleep(1000)` đều `SLEEPING`; con bị `kill` thoát với mã -1	5
10	`RUNNING` / `RUNNABLE`	4 con chạy vòng lặp vô hạn (nhiều hơn số CPU): luôn ở `RUNNING` hoặc `RUNNABLE`, có lúc thấy `RUNNABLE`; `nsched` của mỗi con tăng dần	5
11	Đếm `nsched`	`nsched` của chính chương trình tăng sau `sleep`; con chờ pipe không tăng; con trong `sleep()` vẫn tăng	4
12	Nhiều process con	5 con: `nproc` tăng 5, mọi con có `ppid` đúng; pipe dùng chung có `ref = 6`	6

#	Kịch bản	Nội dung kiểm tra chính	Số KT
13	Process mồ côi	Cha của cháu thoát trước: cháu được chuyển cho `init` (`ppid = 1`), `init` thu dọn cháu khi nó thoát	5
14	Giới hạn `NOFILE`	Mở tới khi `open` lỗi: process có đúng 16 fd, từ 0 tới 15	5
15	Giới hạn `NFILE`	10 process con mở file tới khi lỗi: `nfile` đạt 100; `open` lỗi do `ftable` đầy dù process còn fd trống	6
16	Giới hạn `NPROC`	Fork tới khi lỗi: tạo được 61 con, `nproc` đạt 64	4
17	Con trỏ không hợp lệ	Địa chỉ 0 (trang code chỉ đọc), vượt `MAXVA`, trang `TRAPFRAME`, guard page của stack, buffer tràn qua cuối heap: đều trả về -1; buffer cấp bằng `malloc` hoạt động bình thường	9
		Tổng	**118**

6.3 Một số kết quả tiêu biểu

Các hình dưới đây là output thật của lần chạy với 3 CPU; dòng `...` là chỗ đã lược bớt.

Trạng thái ban đầu (test 1). Chỉ có một file đang mở là console, được 3 process dùng chung với `ref = 9` (mỗi process 3 fd, xem mục 3.3). `sysinfotest` là process duy nhất đang `RUNNING` vì nó chính là process đang gọi `sysinfo`.

```
== test 1: baseline: init, sh, sysinfotest, console ==
  open files: 1
    idx  type    mode  ref  pids
    0    DEVICE  rw    9    1 2 3
  processes: 3
    pid  ppid  state     nsched  name          fd->idx
    1    0     SLEEPING  24      init          0->0 1->0 2->0
    2    1     SLEEPING  11      sh            0->0 1->0 2->0
    3    2     RUNNING   48      sysinfotest   0->0 1->0 2->0
  [PASS] file list and process list agree with each other
  [PASS] ref of every file = number of fds pointing to it
  ...
  [PASS] state of sh (blocked in wait) = SLEEPING
  [PASS] fd 0, 1, 2 share one struct file (init used dup)
```

Hình 2: Kết quả test 1: danh sách open file và process lúc bắt đầu

`fork` (test 5). Con (pid 4) có `ppid = 3`, có đúng các fd của cha và trỏ tới cùng các file. Con đang `RUNNABLE` với `nsched = 0` vì chưa được chọn chạy lần nào.

```
== test 5: fork() ==
  processes: 4
    pid  ppid  state     nsched  name          fd->idx
    1    0     SLEEPING  24      init          0->0 1->0 2->0
    2    1     SLEEPING  11      sh            0->0 1->0 2->0
    3    2     RUNNING   50      sysinfotest   0->0 1->0 2->0 3->1 4->2 5->3
    4    3     RUNNABLE  0       sysinfotest   0->0 1->0 2->0 3->1 4->2 5->3
  [PASS] nproc increase = 1
  [PASS] nfile increase (fork creates no struct file) = 0
  ...
  [PASS] child has the same fds, pointing to the same struct file
  [PASS] each file gains one ref per inherited fd, pids has parent and child
  [PASS] console ref increase (fd 0, 1, 2 inherited) = 3
  ...
  [PASS] offset is shared with the child (child read 'a', parent reads 'b')
```

Hình 3: Kết quả test 5: process con ngay sau `fork`

Trạng thái chạy và `nsched` (test 10, 11). Bốn con chạy vòng lặp vô hạn trên 3 CPU nên luôn có con chờ ở `RUNNABLE`, và `nsched` của chúng tăng sau mỗi lần bị ngắt timer rồi được chọn lại. Con chờ trên pipe rỗng không được đánh thức nên `nsched` đứng yên.

```
== test 10: RUNNING / RUNNABLE spinners ==
    pid  ppid  state     nsched  name          fd->idx
    ...
    9    3     RUNNABLE  1       sysinfotest   0->0 1->0 2->0
    10   3     RUNNABLE  1       sysinfotest   0->0 1->0 2->0
    11   3     RUNNING   2       sysinfotest   0->0 1->0 2->0
    12   3     RUNNING   1       sysinfotest   0->0 1->0 2->0
  [PASS] spinning children are always RUNNING or RUNNABLE
  [PASS] with more spinners than CPUs some of them wait as RUNNABLE
         pid 9: nsched 2 -> 5
         pid 10: nsched 1 -> 6
  ...
== test 11: nsched counting ==
  [PASS] nsched of sysinfotest grows after sleep(2): 67 -> 69
  [PASS] nsched of child blocked on a pipe does not change: 1 -> 1
  [PASS] nsched of child in sleep() grows (woken at every tick): 1 -> 5
```

Hình 4: Kết quả test 10 và 11: trạng thái chạy và bộ đếm `nsched`

Các test còn lại cho kết quả đúng như bảng ở trên: process `ZOMBIE` không còn fd và biến mất sau `wait`, process mồ côi được chuyển cho `init`, các bảng đầy (`NOFILE`, `NFILE`, `NPROC`) vẫn được liệt kê đầy đủ, và mọi địa chỉ user không hợp lệ đều làm `sysinfo` trả về -1 mà không làm hỏng kernel hay process gọi.

7 Khó khăn và các cách đã thử

1. **Phần của A cần `proccollect()` của C để build được.** Trong lúc C chưa xong, A khai báo tạm một `proccollect()` dạng `__attribute__((weak))` trả về 0 trong `sysproc.c`, nhờ vậy vẫn build và thử được phần file. Khi ghép code, hàm thật trong `proc.c` thay thế stub, và nhóm đã xóa stub khỏi bản nộp.

2. **Con đang `sleep()` luôn bị thấy là `RUNNABLE`.** Bản đầu của test 9 chờ bằng cách gọi `sysinfo` rồi `sleep(1)`, và lần nào cũng thấy con trong `sleep(1000)` ở trạng thái `RUNNABLE`. Nguyên nhân: `sys_sleep` ngủ trên kênh `&ticks`, và mỗi tick `clockintr()` đánh thức mọi process ngủ trên kênh này, cả cha lẫn con; cha vừa thức dậy gọi `sysinfo` đúng lúc con cũng vừa được đánh thức. Nhóm sửa bằng cách gọi `sysinfo` liên tục không ngủ, giới hạn thời gian bằng `uptime()`. Từ lỗi này nhóm cũng hiểu vì sao `nsched` của process trong `sleep(n)` vẫn tăng mỗi tick (test 11), còn process chờ pipe thì không.

3. **Test 8 thỉnh thoảng thấy process `ZOMBIE` vẫn còn giữ file.** Có lần chạy chỉ đạt 117/118: con đã là `ZOMBIE`, không còn fd nào, nhưng pid của nó vẫn nằm trong danh sách process của console. Lúc `filecollect()` chạy, con vẫn đang trong `exit()` và còn giữ fd 0, 1, 2; tới lúc `proccollect()` chạy thì con đã đóng hết fd và thành `ZOMBIE`. Đây là lỗi của test, không phải của kernel: sau khi thấy `ZOMBIE`, test giờ gọi `sysinfo` thêm một lần rồi mới kiểm tra, vì `ZOMBIE` không thay đổi gì cho tới khi cha `wait`.

4. **Máy không có sẵn trình biên dịch RISC-V và QEMU.** Nhóm dựng một Docker image Ubuntu 24.04 có `gcc-riscv64-linux-gnu` và `qemu-system-misc` để `make qemu`, và dùng `expect` để tự gõ `sysinfotest` vào shell của xv6, nhờ vậy chạy lại toàn bộ test nhiều lần.

5. **`struct sysinfo` quá lớn để dựng trong kernel.** Struct khoảng 38.5 KB, trong khi kernel stack chỉ 4 KB. Nhóm đã cân nhắc hai cách khác trước khi chọn `copyout` từng phần tử (mục 4.1). Dùng một biến `static` toàn cục thì tránh được tràn stack, nhưng nhiều CPU gọi `sysinfo` cùng lúc sẽ ghi đè lên nhau, phải thêm một khóa lớn. Cấp bằng `kalloc()` thì không được, vì `kalloc()` chỉ cấp từng trang 4 KB, không cấp được khoảng 10 trang liên tiếp.

8 Những khái niệm chưa hiểu rõ

1. **Lấy một bản chụp nhất quán của cả hệ thống.** `sysinfo` chỉ cho kết quả khớp nhau khi hệ thống đang yên (mục 5). Nhóm chưa rõ một hệ điều hành thật như Linux làm thế nào để liệt kê process và file đang mở một cách nhất quán mà không phải dừng mọi CPU hay giữ cùng lúc nhiều khóa, ví dụ các kỹ thuật như RCU giúp được tới đâu.

2. **Khi nào `kill()` thật sự có hiệu lực.** `kill()` chỉ đặt `p->killed` (và đánh thức process nếu nó đang `SLEEPING`); process chỉ gọi `exit()` ở lần trap tiếp theo, khi `usertrap()` kiểm tra cờ này. Nhóm chưa rõ một process đang `RUNNING` trên CPU khác phải chờ tối đa bao lâu mới dừng, và nếu nó đang ở giữa một system call dài (ví dụ chờ đọc đĩa) thì có dừng ngay không.

3. **Khóa được lấy ở thread này và nhả ở thread khác.** Nhóm hiểu vì sao `p->lock` phải được giữ qua `swtch()` (mục 3.6), nhưng chưa hiểu hết cách `noff` và `intena` của CPU vẫn đúng sau khi `swtch()` đổi sang một kernel stack khác, nhất là khi process được chạy tiếp trên một CPU khác với CPU đã lấy khóa.

Tài liệu tham khảo

[1] R. Cox, F. Kaashoek, and R. Morris, "xv6: a simple, Unix-like teaching operating system." https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf, 2024. Bản ngày 31/08/2024; chương 1 (file descriptor, fork), chương 7 (lập lịch), chương 8 (hệ thống file).

[2] "Operating system – Project 1: File & Process Management." Đề bài đồ án môn Hệ điều hành, Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM, 2026.